Ekspertski sistem za procenu cene softverskog rešenja

Aleksandar Stanković
SV25/2022

Domenski ekspert:
Brat, suvlasnik i CEO IT kompanije u Novom Sadu

May 1, 2026

1 Motivacija

Procena cene softverskog rešenja je jedna od najosetljivijih aktivnosti u radu softverske kompanije. Previsoka cena može da odbije potencijalnog klijenta, preniska cena može da dovede do neprofitabilnog projekta, a nejasno obrazložena procena često stvara nepoverenje između klijenta i izvođača. U praksi se inicijalne ponude često prave pod vremenskim pritiskom, na osnovu nepotpunih zahteva, subjektivnog iskustva pojedinih zaposlenih i procena iz prethodnih projekata koje nisu uvek formalizovane.

Tema projekta je ekspertski sistem koji pomaže pri formiranju okvirne cene, odnosno intervala cene, za razvoj softverskog rešenja. Sistem ne treba da zameni konačnu poslovnu odluku, pregovore ili ugovor, već da služi kao objašnjiv alat za podršku proceni. Domensko znanje je modelovano iz prakse procene softverskih projekata: pregovora sa klijentima, planiranja tima, određivanja rizika i izbora odgovarajućeg komercijalnog modela.

Ova tema je pogodna za sistem baziran na znanju zato što proces procene nije čisto matematički problem. Cena zavisi od mnoštva međusobno povezanih faktora: vrste projekta, stepena razrađenosti zahteva, broja i složenosti funkcionalnosti, integracija sa eksternim sistemima, nefunkcionalnih zahteva, roka, raspoloživih resursa, senioriteta tima, tehničkog rizika, komunikacije sa klijentom i poslovnih pravila kompanije. Veliki deo tog znanja već postoji kod domenskog eksperta, ali nije eksplicitno formalizovan.

2 Pregled problema

U literaturi i praksi postoji više metoda za procenu softverskih projekata. COCOMO II procenjuje napor na osnovu veličine softvera i faktora projekta. Function Point Analysis pokušava da proceni veličinu sistema kroz funkcionalne tačke, dok Use Case Points polaze od aktera, slučajeva korišćenja i tehničkih faktora. U agilnim timovima se često koriste story points, velocity i ekspertska procena tima. Komercijalni alati za upravljanje projektima, prodajom i ponudama obično omogućavaju unos stavki ponude, satnica, resursa i popusta.

Problem postojećih pristupa u kontekstu malih i srednjih softverskih kompanija jeste to što su često ili previše opšti, ili previše zavisni od prethodno izmerenih metrika koje kompanija nema u dovoljnoj meri. Sa druge strane, ručne tabele i subjektivne procene nisu dovoljno objašnjive, teško ih je održavati i ne reaguju dobro na promene zahteva tokom pregovora. Često se ne vidi zašto je cena povećana, koji rizik najviše utiče na interval cene i koje informacije klijent mora da dostavi da bi se interval suzio.

Predloženi sistem se razlikuje od klasičnog kalkulatora cene po tome što koristi bazu znanja, pravila i objašnjivo rezonovanje. Sistem će:

- klasifikovati zahtev po složenosti, tipu projekta i nivou neizvesnosti;
- izvesti procene napora po oblastima rada i ulogama u timu;
- koristiti pravila domenskog eksperta za rizike, rezerve, popuste, minimalnu cenu i uslove ponude;
- dati interval cene, a ne lažno preciznu jednu vrednost;
- objasniti koji su faktori uticali na cenu;
- pratiti promene zahteva kroz događaje i upozoravati kada trenutna ponuda više nije pouzdana;
- omogućiti proveru ciljeva, na primer da li projekat može da se realizuje u zadatom budžetu ili roku.

3 Opseg projekta

Sistem procenjuje cenu za projekte razvoja softverskih rešenja po meri: veb aplikacije, interne poslovne aplikacije, SaaS MVP proizvode, nadogradnje postojećeg sistema, integracije sa eksternim servisima i migracije podataka.

Izlaz sistema biće okvirna ponuda sa intervalom cene, preporučenim timom, okvirnim trajanjem, pretpostavkama i objašnjenjem. Konačna komercijalna ponuda i ugovor ostaju odgovornost ovlašćenog zaposlenog u kompaniji.

4 Korisnici sistema

Aplikacija će podržavati tri tipa korisnika:

- **Estimator / sales korisnik** – unosi zahtev klijenta, pokreće rezonovanje, pregleda cenu, objašnjenja, rizike i predloge za sužavanje opsega.
- **Domenski ekspert / administrator** – održava bazu znanja, menja tabele za generisanje pravila, unosi satnice, pragove rizika, tipove funkcionalnosti, pravila za popuste i minimalne margine.
- **Klijent ili predstavnik klijenta** – popunjava inicijalni upitnik o projektu i dostavlja dodatne informacije tokom discovery faze. Klijent ne menja pravila, već samo unosi činjenice koje sistem koristi.

5 Očekivani ulazi u sistem

Ulazi u sistem obuhvataju statičke podatke o zahtevu, podatke o resursima kompanije i događaje koji nastaju tokom komunikacije sa klijentom.

5.1 Podaci o klijentu i projektu

- tip klijenta: startup, mala firma, srednja firma, enterprise, javni sektor;
- domen projekta: e-commerce, interna administracija, SaaS, integracioni sistem, analitika, automatizacija procesa, drugo;
- tip angažmana: MVP, nova aplikacija, nadogradnja, refaktorisanje, migracija, održavanje;
- cilj projekta i prioriteti: brz izlazak na tržište, stabilnost, skalabilnost, sigurnost, integracije, niska cena;

- budžetski opseg ako ga klijent zna;
- željeni rok i kritičnost roka;
- nivo razrađenosti zahteva: ideja, opis procesa, backlog, wireframe, Figma dizajn, tehnička specifikacija;
- dostupnost klijenta za radionice i odluke.

5.2 Funkcionalni zahtevi

Za svaki zahtev ili modul unose se:
- naziv i opis funkcionalnosti;
- tip funkcionalnosti: CRUD modul, workflow, dashboard, izveštavanje, plaćanje, notifikacije, administracija, autentifikacija, integracija, migracija, real-time komunikacija, AI/automatizacija, drugo;
- broj korisničkih uloga koje koriste funkcionalnost;
- broj poslovnih pravila u funkcionalnosti;
- da li postoji dizajn ekrana;
- broj ekrana ili korisničkih tokova;
- zavisnosti od drugih funkcionalnosti;
- prioritet: must-have, should-have, could-have;
- da li postoji nepoznat deo zahteva.

5.3 Integracije i tehnički zahtevi

- broj eksternih integracija;
- tip integracije: REST API, SOAP, webhook, payment provider, ERP/CRM, custom API, legacy baza;
- dostupnost dokumentacije, sandbox okruženja i testnih kredencijala;
- da li integracija zahteva asinhronu obradu, retry mehanizam ili audit log;
- migracija podataka: format izvora, količina podataka, čišćenje podataka, mapiranje polja;
- nefunkcionalni zahtevi: sigurnost, performanse, dostupnost, skalabilnost, observability, compliance, SLA;
- preferirani tehnološki stek ako ga klijent ima.

5.4 Resursi i poslovna pravila kompanije

- dnevne satnice po ulozi: backend, frontend, QA, UX/UI, DevOps, PM/BA, arhitekta;
- minimalna marža i minimalna prihvatljiva cena projekta;
- raspoloživost tima u narednim nedeljama;
- senioritet članova tima;
- pravila za popust, hitan rok, fiksnu cenu i time-and-materials model;
- istorijski podaci o sličnim projektima, ukoliko su dostupni.

5.5 Događaji za CEP

Tokom discovery procesa sistem prima događaje:
- dodat novi zahtev;
- izmenjen postojeći zahtev;
- obrisan zahtev;
- klijent poslao odgovor na otvoreno pitanje;

- klijent kasni sa odgovorom;
- dostavljena ili odbijena dokumentacija za integraciju;
- promenjen rok;
- promenjen budžet;
- poslata ponuda;
- klijent odbio ponudu zbog cene, roka ili opsega;
- održana radionica ili tehnički sastanak.

6 Očekivani izlazi iz sistema

Glavni izlazi iz sistema su:

- procenjeni interval cene, na primer 18.197–24.262–31.541 EURza inicijalni SaaS MVP discovery interval, 65.611–77.516–87.071 EURza stabilniji partner extension ili 17.594–23.458–30.495 EURza plaćenu discovery fazu visokorizičnog compliance projekta;
- očekivani, minimalni i maksimalni napor po ulozi;
- preporučeni tim, na primer 1 senior backend, 1 medior frontend, 0.5 QA, 0.25 PM;
- okvirno trajanje projekta;
- nivo pouzdanosti procene: visok, srednji, nizak;
- lista pretpostavki pod kojima cena važi;
- lista nedostajućih informacija;
- rizici koji utiču na interval cene i procenat rezerve;
- preporuka modela saradnje: fiksna cena, time-and-materials, plaćena discovery faza, fazna isporuka;
- objašnjenje aktiviranih pravila;
- upozorenja iz CEP modula kada se zahtev menja prebrzo ili kada ponuda treba da se revidira;
- odgovor na ciljni upit: da li je projekat izvodljiv u zadatom budžetu ili roku.

7 Model podataka

Planirani model podataka biće prilagođen implementaciji u Java/Spring aplikaciji i Drools pravilima. Osnovni entiteti su:

Entitet	**Opis**
`ClientProfile`	Podaci o tipu klijenta, domenu, zrelosti klijenta, budžetu i načinu donošenja odluka.
`ProjectRequest`	Glavni zahtev za procenu: tip projekta, rok, prioriteti, nivo specifikacije, željeni model saradnje.
`Requirement`	Jedan funkcionalni zahtev ili modul, sa tipom, prioritetom, složenošću, brojem ekrana i zavisnostima.
`IntegrationRequirement`	Poseban zahtev za integraciju sa eksternim sistemom, sa informacijama o dokumentaciji, sandbox-u i tipu API-ja.
`NonFunctionalRequirement`	Zahtevi za performanse, sigurnost, dostupnost, skalabilnost, audit, SLA i compliance.
`MissingInformation`	Informacija koja nedostaje da bi procena bila pouzdanija.
`ComplexityTag`	Izvedena činjenica koja označava složenost zahteva, integracije ili projekta.

Entitet	Opis
`WorkPackageEstimate`	Izvedena procena napora po oblasti rada i ulozi, na primer backend 8 dana za payment integraciju.
`RoleRate`	Dnevna satnica po ulozi i senioritetu.
`TeamPlan`	Izveden predlog sastava tima i trajanja projekta.
`RiskFactor`	Izvedeni ili ručno uneti rizik sa težinom i procentom rezerve.
`QuoteRange`	Konačni rezultat rezonovanja: minimalna, očekivana i maksimalna cena, uz nivo pouzdanosti.
`ReasoningTrace`	Objašnjenje koji su ulazi i pravila doveli do procene.
`DiscoveryEvent`	Apstraktna klasa događaja za CEP, sa timestamp-om, projektom i tipom događaja.

8 Baza znanja

Baza znanja će se puniti iz četiri izvora:

1. intervju sa domenskim ekspertom, kroz koji se formalizuju pravila za procenu složenosti, rezerve, popuste i poslovne odluke;
2. anonimni primeri prethodnih procena i projekata iz kompanije, ukoliko budu dostupni;
3. opšte poznate metode procene softvera, kao što su COCOMO II, Function Point Analysis, Use Case Points i agilne procene;
4. konfiguracione tabele koje administrator može da menja bez izmene programskog koda.

Znanje nije zapisano samo u kodu aplikacije. Deo pravila je pisan kao `.drl` pravila, dok se deo znanja održava kroz `.drt` template-e i tabele. Implementacija učitava redove template tabela kao činjenice koje koriste pravila, a aplikacija može da prikaže generisani DRL pregled za proveru od strane domenskog eksperta. Time se omogućava da domenski ekspert menja pragove, satnice i faktore bez potrebe za promenom Java koda.

8.1 Grupe pravila

Baza znanja biće podeljena u sledeće grupe pravila:

1. **Pravila za validaciju i dopunu zahteva** – proveravaju da li su navedeni cilj projekta, rok, prioriteti, osnovni moduli, integracije i nivo specifikacije.
2. **Pravila za klasifikaciju složenosti** – određuju složenost pojedinačnih funkcionalnosti, integracija, migracija i celog projekta.
3. **Pravila za procenu napora** – na osnovu tipa i složenosti zahteva generišu `WorkPackageEstimate` činjenice po ulozi.
4. **Pravila za agregaciju napora i formiranje tima** – koriste `accumulate` za sabiranje dana po ulozi i predlažu sastav tima.
5. **Pravila za rizik i pouzdanost procene** – dodaju rezerve za nejasne zahteve, lošu dokumentaciju, kratke rokove, visoke nefunkcionalne zahteve i zavisnost od trećih strana.
6. **Pravila za cenu i komercijalne uslove** – računaju osnovnu cenu, interval cene, popuste, minimalnu cenu i preporučeni model saradnje.
7. **CEP pravila** – prate tok discovery procesa i zaključuju kada je ponuda zastarela, kada raste scope creep ili kada treba predložiti plaćenu discovery fazu.
8. **Backward chaining upiti** – proveravaju izvodljivost ciljeva i pronalaze nedostajuće informacije ili zavisnosti.

9. **Izveštaji i query pravila** – omogućavaju pretragu procena, poređenje sa sličnim projektima i pregled najčešćih rizika.

9 Primeri pravila

U nastavku su opisani primeri pravila. Tokom implementacije pravila mogu biti dopunjena u skladu sa dodatnim informacijama domenskog eksperta.

9.1 Pravila za validaciju zahteva

- Ako projekat nema opisan poslovni cilj, sistem dodaje `MissingInformation` sa visokim prioritetom i snižava pouzdanost procene.
- Ako klijent traži fiksnu cenu, a nema definisane must-have funkcionalnosti, sistem preporučuje plaćenu discovery fazu pre fiksne ponude.
- Ako je rok kraći od minimalnog roka za procenjeni obim, sistem dodaje `RiskFactor` tipa `RUSH_DEADLINE`.
- Ako funkcionalnost nema prioritet, podrazumeva se `SHOULD_HAVE`, ali se dodaje napomena da prioritet mora biti potvrđen pre finalne ponude.

9.2 Pravila za složenost funkcionalnosti

- CRUD modul sa do 4 ekrana, najviše 2 korisničke uloge i bez složenog workflow-a dobija složenost `LOW`.
- Modul sa 5–10 ekrana, 3 ili više korisničkih uloga ili više stanja procesa dobija složenost `MEDIUM`.
- Modul sa više od 10 ekrana, kompleksnim workflow-om, naprednim pravima pristupa ili audit logom dobija složenost `HIGH`.
- Payment integracija sa poznatim provider-om, dobrom dokumentacijom i sandbox-om dobija složenost `MEDIUM`.
- Integracija bez dokumentacije ili bez testnog okruženja dobija složenost `HIGH` i rizik `THIRD_PARTY_UNCERTAINTY`.
- Migracija podataka dobija složenost `HIGH` ako podaci dolaze iz nestrukturisanih fajlova ili ako je potrebno čišćenje i deduplikacija.

9.3 Pravila za procenu napora

Procena napora biće generisana kombinacijom ručno pisanih pravila i template-a. Primeri:

- Ako je zahtev tipa `CRUD` i složenosti `LOW`, ubacuju se procene za frontend, backend i QA sa malim opsegom dana.
- Ako zahtev ima audit log, ubacuje se dodatna backend procena i dodatna QA procena.
- Ako projekat ima više od 3 korisničke uloge, dodaje se procena za detaljnije testiranje prava pristupa.
- Ako postoji Figma dizajn, smanjuje se UX/UI napor, ali se ne smanjuje frontend napor za kompleksne interakcije.
- Ako nema dizajna, sistem dodaje UX/UI work package i `MissingInformation` za vizuelni identitet.

9.4 Pravila za rizik

- Ako je nivo specifikacije samo ideja, osnovna rezerva je najmanje 25%.
- Ako postoji backlog i prioriteti su označeni, osnovna rezerva je 10–15%.
- Ako postoje finalni dizajn, tehnička specifikacija i dostupni testni nalozi, osnovna rezerva može biti 5–10%.
- Ako postoji više od 2 integracije sa trećim stranama, dodaje se dodatna rezerva od 5–15%, zavisno od kvaliteta dokumentacije.
- Ako klijent zahteva rok koji je kraći za više od 20% od procenjenog trajanja, dodaje se `RUSH_PREMIUM` i preporučuje fazna isporuka.
- Ako se očekuje visok nivo compliance-a, audit loga ili sigurnosnih zahteva, dodaje se arhitektonska i QA rezerva.

9.5 Pravila za cenu

Osnovna cena računa se kao suma prodajnih cena po ulogama:

$$osnovnaCena = \sum_{uloga} brojDana(uloga) \times dnevnaCena(uloga)$$

Donja granica cene uključuje minimalnu maržu i kontrolisanu rezervu, dok gornja granica uključuje dodatnu rezervu za neizvesnost.
Primeri poslovnih pravila:

- Ako je izračunata cena manja od minimalne cene projekta, postavlja se minimalna cena i dodaje objašnjenje.
- Ako je procena niske pouzdanosti, sistem preporučuje `PAID_DISCOVERY` i prikazuje manji discovery interval, tipično oko 18% očekivane cene punog projekta, ograničen na razuman opseg za demo i realnu prodajnu praksu.
- Ako je budžet klijenta manji od donje granice cene, sistem predlaže smanjenje opsega ili MVP varijantu.
- Ako je budžet unutar intervala, ali je rok visokorizičan, sistem ne odbija projekat već predlaže veći tim ili faznu isporuku.

10 Forward chaining rezonovanje

U opštem slučaju lanac rezonovanja izgleda ovako:

1. **Početne činjenice**: korisnik unosi `ProjectRequest`, `Requirement`, `IntegrationRequirement`, `NonFunctionalRequirement`, `ClientProfile` i `RoleRate`.
2. **Nivo 1 – klasifikacija**: pravila ubacuju `ComplexityTag`, `MissingInformation` i početne `RiskFactor` činjenice.
3. **Nivo 2 – procena pojedinačnih stavki**: pravila na osnovu složenosti ubacuju `WorkPackageEstimat` činjenice po ulozi.
4. **Nivo 3 – agregacija**: koriste se `accumulate` funkcije da se izračuna ukupan broj dana po ulozi i ukupan broj rizičnih ili nejasnih stavki.
5. **Nivo 4 – plan tima i trajanje**: na osnovu agregiranih dana ubacuju se `DeliveryEstimate`, `TeamPlan` i alokacije po ulogama.
6. **Nivo 5 – cena i interval**: pravila računaju osnovnu cenu, donju i gornju granicu, pouzdanost i preporučeni model saradnje. Cena koristi agregirane cene po ulozi, ukupan rizik, broj nedostajućih informacija, pricing policy i FTE-plan tima.

7. **Nivo 6 – objašnjenje i preporuke**: pravila ubacuju `ReasoningTrace`, predlog MVP smanjenja opsega i listu pitanja za klijenta.
   Primer upotrebe `accumulate` funkcije:

```
rule "Aggregate backend effort"
when
    $project : ProjectRequest($id : id)
    $backendDays : Number() from accumulate(
        WorkPackageEstimate(projectId == $id,
                            role == Role.BACKEND,
                            $d : expectedDays),
        sum($d)
    )
then
    insert(new AggregatedEffort($id, Role.BACKEND,
                                $backendDays.doubleValue()));
end
```

Slične agregacije koriste se za minimalne i maksimalne dane, broj nejasnih zahteva, broj integracija bez dokumentacije, ukupnu težinu rizika, cenu po ulozi i broj događaja u CEP vremenskim prozorima.

11 Rule template-i

Template-i su važni zato što se mnoga pravila zasnivaju na tabelama koje domenski ekspert treba da održava. Implementacija ima `.drt` fajlove i tabele u bazi za procene zahteva i rezerve rizika. Redovi tabela se u operativnoj Drools sesiji ubacuju kao činjenice, dok se isti redovi mogu prikazati kroz generisani DRL preview iz `.drt` fajlova radi provere.

11.1 Template za procenu funkcionalnosti

Primer tabele iz koje se održavaju procene i prikazuje DRL preview:

Tip zahteva	Složenost	Frontend	Backend	QA	Napomena
CRUD modul	LOW	2–3	2–4	1	Jednostavna administracija
CRUD modul	MEDIUM	4–6	5–7	2–3	Više uloga ili validacija
Workflow	MEDIUM	5–8	7–10	3–4	Više stanja procesa
Workflow	HIGH	9–14	12–18	5–7	Kompleksna odobrenja
Dashboard	MEDIUM	4–7	3–6	2–3	Filtri i grafikoni
Payment	MEDIUM	3–5	6–9	3–5	Provider sa dokumentacijom

Tip zahteva	Složenost	Frontend	Backend	QA	Napomena
Integracija	HIGH	2–4	8–15	4–7	Nepotpuna dokumentacija

Svaki red predstavlja znanje koje se u implementaciji učitava kao `RequirementEstimateTemplateRow` činjenica, a isti podaci mogu da se prikažu i kao generisani DRL preview. Domenski ekspert može kasnije da promeni brojeve bez promene Java koda.

11.2 Template za rizike

Drugi template održava pravila za rizik:

Uslov	Rizik	Rezerva	Preporuka
Specifikacija je samo ideja	UNCLEAR_SCOPE	25%	Plaćena discovery faza
Nema sandbox za integraciju	THIRD_PARTY	10%	Tehnička radionica
Rok kraći od procene	RUSH	10–20%	Fazna isporuka
Više od 3 korisničke uloge	ACCESS_CONTROL	5%	Dodatni QA
Visok SLA ili compliance	COMPLIANCE	10–20%	Uključiti arhitektu

12 CEP deo projekta

U toku pregovora i discovery faze zahtevi se menjaju kroz vreme. Ti događaji direktno utiču na pouzdanost procene i treba ih obraditi kao tok događaja.

12.1 Tipovi događaja

Događaj	Značenje
`RequirementAddedEvent`	Klijent dodaje novu funkcionalnost.
`RequirementChangedEvent`	Menja se opis, prioritet ili složenost postojećeg zahteva.
`RequirementRemovedEvent`	Funkcionalnost se izbacuje iz opsega.
`ClientAnswerEvent`	Klijent odgovara na otvoreno pitanje.
`ClientDelayEvent`	Klijent ne odgovara u dogovorenom roku.
`DeadlineChangedEvent`	Klijent menja željeni rok.
`IntegrationDocsReceivedEvent`	Dostavljena je dokumentacija za eksternu integraciju.
`QuoteSentEvent`	Estimator šalje ponudu.
`QuoteRejectedEvent`	Klijent odbija ponudu zbog cene, roka ili opsega.

12.2 Primeri CEP pravila

- Ako se u poslednjih 7 dana desi više od 5 događaja izmene ili dodavanja zahteva, sistem zaključuje `HIGH_SCOPE_VOLATILITY` i dodaje rizik od scope creep-a.

- Ako posle poslate ponude u narednih 14 dana budu dodate funkcionalnosti čiji zbirni napor prelazi 20% prethodno procenjenog napora, sistem označava ponudu kao zastarelu.
- Ako klijent tri puta zaredom kasni sa odgovorom na blokirajuće pitanje, sistem snižava pouzdanost procene i predlaže da se otvorena pitanja stave kao eksplicitne pretpostavke.
- Ako se rok skrati dva ili više puta u periodu od 10 dana, sistem dodaje `RUSH_DEADLINE` i predlaže veći tim ili smanjenje opsega.
- Ako se budžet više puta menja u kratkom periodu, sistem dodaje `BUDGET_PRESSURE` i predlaže ponovno definisanje MVP opsega.
- Ako klijent odbije ponudu zbog cene, sistem dodaje `QUOTE_REJECTED_FOR_PRICE` i predlaže faznu isporuku, smanjeni MVP ili discovery.
- Ako discovery teče stabilno, kroz radionice, odgovore klijenta i dostavljenu dokumentaciju, sistem prikazuje `STABLE_DISCOVERY` i preporuku da estimator može suziti interval ako se ne pojave novi visoki rizici.

Primer CEP pravila sa vremenskim prozorom:

```
rule "High scope volatility in discovery"
when
    $project : ProjectRequest($id : id)
    $changes : Number(intValue > 5) from accumulate(
        DiscoveryEvent(projectId == $id,
                       type in (EventType.REQUIREMENT_ADDED,
                                EventType.REQUIREMENT_CHANGED))
            over window:time(7d),
        count(1)
    )
then
    insert(new RiskFactor($id, RiskType.SCOPE_VOLATILITY,
                          0.15,
                          "Vise od 5 promena zahteva u 7 dana"));
    insert(new Recommendation($id,
          "Predloziti placenu discovery fazu ili siru cenu."));
end
```

13 Backward chaining

Backward chaining će se koristiti za proveru ciljeva i objašnjavanje zavisnosti. Umesto da korisnik ručno pretražuje sve razloge zbog kojih projekat ne može da stane u budžet ili rok, korisnik može da postavi cilj, a sistem pokušava da ga dokaže kroz postojeće činjenice i zavisnosti.

13.1 Primeri backward chaining upita

- **Da li projekat može da se realizuje u budžetu?** Sistem proverava da li je gornja granica procene manja od budžeta, da li ne postoje blokirajuće nepoznanice i da li rizik nije previsok.
- **Da li projekat može da se realizuje do zadatog roka?** Sistem proverava procenjeno trajanje, raspoloživost tima, zavisnosti i rizike.
- **Koje informacije nedostaju za pouzdanu ponudu?** Sistem rekurzivno prolazi kroz zahteve i zavisnosti da nađe nedostajuće informacije.

- **Zašto je određena funkcionalnost skupa?** Sistem prolazi kroz zavisnosti funkcionalnosti, integracije, audit log, broj uloga i nefunkcionalne zahteve.

  Primer rekurzivnog upita za zavisnosti:

```
query dependsOn(String requirementId, String dependencyId)
    Dependency(requirementId, dependencyId;)
or
    (Dependency(requirementId, $middle;)
     and dependsOn($middle, dependencyId;))
end
```

  Primer upita za cilj izvodljivosti:

```
query feasibleWithinBudget(String projectId, double budget)
    QuoteRange(projectId == projectId, maxPrice <= budget;)
    not MissingInformation(projectId == projectId,
                           severity == Severity.BLOCKING;)
    not RiskFactor(projectId == projectId,
                   level == RiskLevel.CRITICAL;)
end
```

Ovi upiti će biti dostupni estimatoru kroz ekran za proveru ciljeva. Ako cilj nije dokaziv, sistem prikazuje najvažnije razloge: previsoka cena, blokirajuće nepoznanice, prekratak rok ili previsok rizik.

14 Izveštaji i query pravila

Sistem podržava nekoliko izveštaja i izvoz ponude:
- najčešći rizici u zadatom vremenskom prozoru, sa brojem pojava i prosečnom rezervom;
- prosečne procene po tipu projekta;
- prosečne procene po domenu klijenta;
- proseci po tipu funkcionalnosti;
- procene kod kojih je ponuda zastarela zbog kasnijih discovery događaja;
- lista projekata kod kojih je klijentov budžet bio ispod donje granice procene;
- poređenje aktuelne procene sa sličnim ranijim procenama;
- istorija verzija ponude i snimljeni discovery događaji za jedan projekat;
- Markdown export ponude sa intervalom cene, planom tima, rizicima i preporukama.

Za izveštaje se kombinuju snimljene kalkulacije, perzistirani rizici i live preview rezonovanje nad trenutnim stanjem projekta. Izveštaji nisu glavni deo rezonovanja, ali pomažu domenskom ekspertu da proveri i poboljša pravila.

15 Konkretan primer rezonovanja

Klijent traži SaaS MVP za upravljanje terminima. Projekat treba da ima:

- registraciju, prijavu i tri korisničke uloge;
- CRUD module za korisnike, usluge i termine;
- booking workflow za zakazivanje, pomeranje i otkazivanje termina;
- dashboard sa osnovnim metrikama;
- online plaćanje preko Stripe-a;
- email notifikacije;

- integracije sa Google Calendar API-jem i Postmark-om;
- responzivni veb interfejs i standardnu dostupnost;
- rok od 14 nedelja;
- delimično spreman Figma dizajn;
- budžet 60.000–95.000 EUR.

15.1 Početne činjenice

U radnu memoriju se ubacuju činjenice:

- `ProjectRequest`: tip `MVP`, rok 14 nedelja, specifikacija `PARTIAL_FIGMA`, tražena fiksna cena;
- `ClientProfile(type=startup, domain=appointment scheduling)`;
- `Requirement(type=AUTHENTICATION, roles=3, priority=MUST_HAVE)`;
- `Requirement(type=CRUD, screens=6, rules=8, priority=MUST_HAVE)`;
- `Requirement(type=WORKFLOW, states=3, priority=MUST_HAVE)`;
- `Requirement(type=DASHBOARD, screens=3, priority=SHOULD_HAVE)`;
- `Requirement(type=PAYMENT, provider=STRIPE, priority=MUST_HAVE)`;
- `Requirement(type=NOTIFICATION, priority=SHOULD_HAVE)`;
- integracija `STRIPE`: dokumentacija dostupna, sandbox dostupan;
- integracija `GOOGLE_CALENDAR`: delimična dokumentacija, kredencijali nisu finalizovani;
- integracija `POSTMARK`: delimična dokumentacija, kredencijali nisu finalizovani;
- nefunkcionalni zahtevi `RESPONSIVE_WEB` i `AVAILABILITY`;
- aktivne `RoleRate` činjenice za backend, frontend, QA, DevOps, PM/BA, UX/UI i arhitektu;
- aktivna `PricingPolicy` za startup klijente.

15.2 Nivo 1 – validacija i klasifikacija

Aktiviraju se pravila za validaciju i klasifikaciju:

- delimičan Figma dizajn dodaje `MissingInformation(design.remainingScreens)`;
- `PARTIAL_FIGMA` dodaje osnovnu rezervu za nejasan preostali UI opseg;
- zahtevi dobijaju složenosti `MEDIUM` ili `HIGH` prema tipu, broju ekrana, uloga, poslovnih pravila, audit logu i workflow stanjima;
- integracije dobijaju složenost prema tipu integracije, dokumentaciji, sandbox-u, kredencijalima i potrebi za retry/audit obradom;
- više integracija dodaje koordinacioni i regresioni rizik, čak i kada su delimično dokumentovane.

15.3 Nivo 2 – procena stavki

Template redovi i ručno pisana pravila ubacuju `WorkPackageEstimate` činjenice. Primeri izvedenih stavki:

Stavka	Uloga	Izvor	Razlog
Autentifikacija i uloge	Backend/Frontend/QA	template + add-on	Više uloga i osnovna prava pristupa
CRUD moduli	Backend/Frontend/QA	template	Standardni ekrani, forme i validacije

Stavka	Uloga	Izvor	Razlog
Booking workflow	Backend/QA/PM	pravilo	Stanja procesa i testiranje prelaza
Dashboard	Frontend/Backend	template + add-on	Filtri, metrika i agregacije
Online plaćanje	Backend/QA	template + add-on	Stripe, webhook i finansijski edge-case scenariji
Email notifikacije	Backend/QA	template + add-on	Triggeri, template-i i provera isporuke
Responsive web	Frontend/QA	NFR pravilo	Prilagođavanje i testiranje na više uređaja

15.4 Nivo 3 – agregacija i plan tima

Pravila sa `accumulate` sabiraju minimalne, očekivane i maksimalne dane po ulozi. Zatim se formiraju:

- `AggregatedEffort` po ulozi;
- `QuoteCostByRole` na osnovu dnevnih satnica;
- `AggregatedRisk` sa ukupnom rezervom i brojem visokih rizika;
- `AggregatedInputGaps` sa brojem nedostajućih informacija i problematičnih integracija;
- `DeliveryEstimate` sa očekivanim i maksimalnim trajanjem;
- `TeamPlan` sa frakcionim FTE alokacijama.

15.5 Nivo 4 – rizik i pouzdanost

Za SaaS MVP aktiviraju se sledeći zaključci:

- delimičan dizajn i nefinalizovane kredencijale treba zapisati kao pretpostavke;
- payment i integracije povećavaju backend i QA napor;
- rok od 14 nedelja je kraći od maksimalne procene isporuke, pa se dodaje rizik roka;
- fiksna cena je tražena pre nego što su svi ulazi dovoljno stabilni;
- zbog kombinacije rizika, sistem snižava pouzdanost na `LOW` i ne zaključava punu implementaciju kao fiksnu cenu.

15.6 Nivo 5 – cena i ponuda

Na osnovu prethodno izvedenih činjenica sistem formira opseg cene i preporuku modela saradnje. Za dati scenario aplikacija prikazuje:

$$\boxed{18.197 - 24.262 - 31.541 \text{ EUR}}$$

Prva vrednost predstavlja donju granicu procene, druga očekivanu cenu, a treća gornju granicu procene za preporučenu plaćenu discovery fazu. Pouzdanost procene je `LOW`, preporučeni model saradnje je `PAID_DISCOVERY`, dok je objekat `QuoteRange` označen sa `narrowedToDiscover` To znači da sistem ne preporučuje izdavanje fiksne ponude za kompletan MVP na osnovu trenutnih, još uvek rizičnih pretpostavki. Umesto toga, sistem predlaže ograničenu plaćenu

discovery fazu u kojoj se finalizuju dizajn, integracije, otvorena pitanja i realan obim prve isporuke.

Ovakav rezultat je važan zato što sistem ne pokušava da prikaže lažno preciznu cenu celog projekta. Klijent je naveo okvirni budžet od 60.000 do 95.000 EUR, ali zbog nepotpunog dizajna, nedostajućih informacija o integracijama i povećanog rizika roka, sistem zaključuje da puna ponuda trenutno nije dovoljno pouzdana. Discovery faza zato služi kao bezbedniji komercijalni model: kompanija naplaćuje analizu i specifikaciju, a nakon smanjenja neizvesnosti može da izda pouzdaniju ponudu za implementaciju kompletnog MVP-a.

Sistem generiše sledeće objašnjenje:

- najveći deo pune procene dolazi iz backend, frontend i QA rada na MVP funkcionalnostima;
- payment, workflow i integracije povećavaju napor, tehnički rizik i potrebnu rezervu;
- lista nedostajućih informacija pokazuje šta mora biti potvrđeno pre pouzdane pune ponude;
- zbog niske pouzdanosti sistem ne preporučuje fiksnu cenu za ceo MVP u ovoj fazi;
- preporuke uključuju `ATTACH_ASSUMPTIONS`, `NARROW_TO_PAID_DISCOVERY` i faznu isporuku nakon završetka discovery faze.

Konačan zaključak sistema je da projekat potencijalno može da se uklopi u navedeni budžetski opseg, ali da trenutni nivo neizvesnosti ne dozvoljava odgovorno zaključavanje pune cene. Zato se kao prvi korak predlaže plaćena discovery faza, nakon koje se ponovo pokreće rezonovanje nad preciznijim zahtevima i formira konačna ponuda za implementaciju.

References

[1] B. Boehm et al., *Software Cost Estimation with COCOMO II*, Prentice Hall, 2000.

[2] International Function Point Users Group, *Function Point Counting Practices Manual.*

[3] G. Karner, *Resource Estimation for Objectory Projects*, Objective Systems, 1993.

[4] M. Cohn, *Agile Estimating and Planning*, Prentice Hall, 2005.

[5] Drools Documentation, *Drools Rule Engine, CEP, Rule Templates and Queries.*